

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 — Introduction to Artificial Intelligence

PRN: _____ (fill in)

Name: _____ (fill in)

Division: A / B (circle one)

Date: _____ (fill in)

GitHub / Portfolio Link (optional): _____

1. System Title & Short Description

Maze Solver & Profiling System (BFS vs DFS)

This is the same system built and profiled in SLE-2. It generates a 15×15 perfect maze with a randomized recursive-backtracker and solves it from cell (0,0) to cell (14,14) using two uninformed search agents: Breadth-First Search (BFS) and Depth-First Search (DFS). A profiling harness then runs each algorithm 50 times and records best, average and worst time plus nodes expanded and path length. The problem it solves is simple: deciding, with measured data rather than theory alone, which search strategy suits a given maze. SLE-1 gave the code, SLE-2 measured its performance, and this SLE-3 documents its architecture.

2. Context Diagram (Level 1)

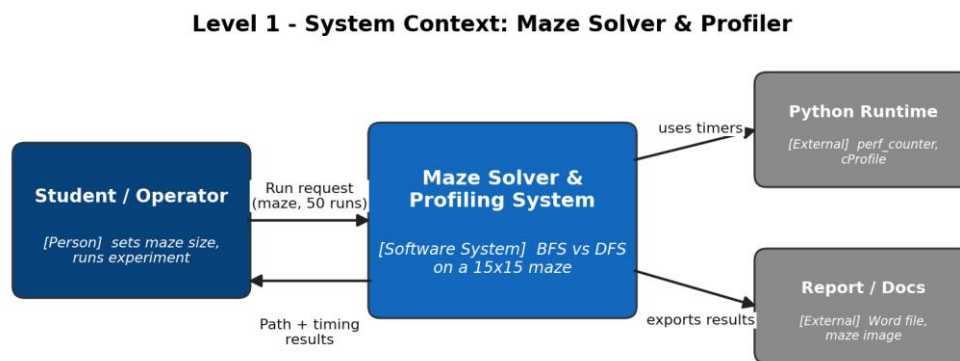


Figure 1: System context of the Maze Solver & Profiling System

The Student / Operator is the only human user. They give a run request (maze, number of runs) to the system and get back the solved path and the timing results. The system depends on the Python runtime, namely `time.perf_counter` and `cProfile`, to measure time. Results are exported to the report (Word file and maze image). No network or database is involved, so the context stays small.

3. Container Diagram (Level 2)

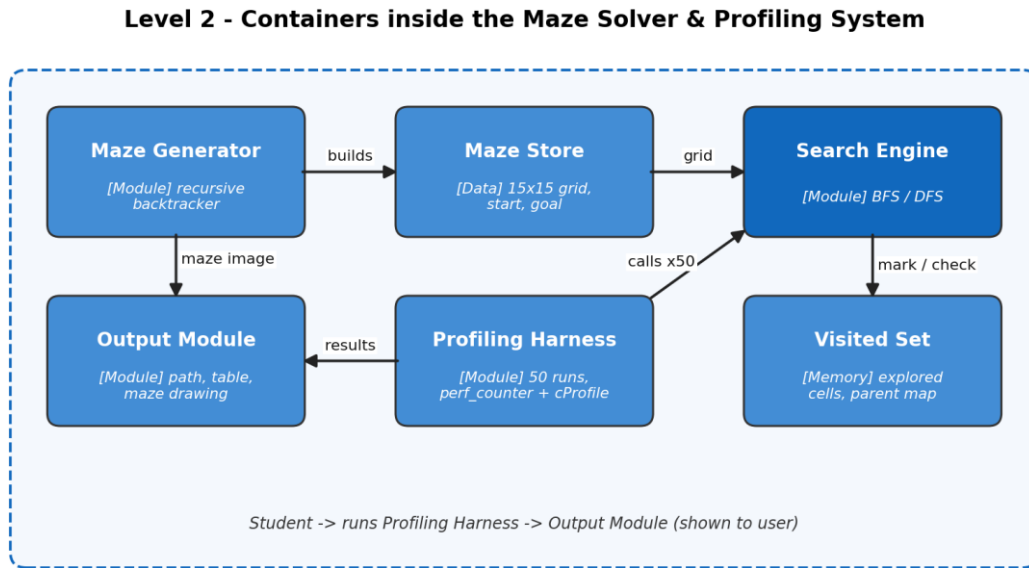


Figure 2: Containers (six building blocks) inside the system

- **Maze Generator:** creates the random 15×15 perfect maze once and keeps it fixed for all runs.
- **Maze Store:** holds the grid, start and goal; the single shared input for both algorithms.
- **Search Engine:** runs BFS or DFS on the stored maze and returns the path and nodes expanded.
- **Visited Set:** remembers explored cells and each cell's parent, preventing loops and enabling path recovery.
- **Profiling Harness:** calls the Search Engine 50 times per algorithm and records the timings.
- **Output Module:** prints the comparison table and draws the maze with the solved path.

4. Component Diagram (Level 3)

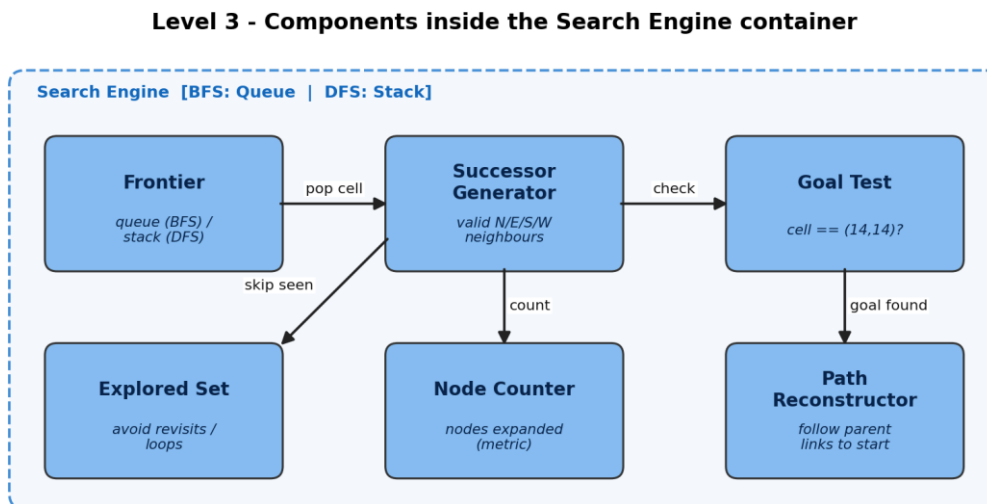


Figure 3: Components inside the Search Engine container

The Search Engine was chosen because it is the heart of the system. The Frontier is the only part that differs between algorithms (a queue for BFS, a stack for DFS). Each popped cell goes to the Successor Generator, which returns valid neighbours; the Explored Set filters out cells already seen. The Goal Test stops the search at (14,14), the Node Counter records how many cells were expanded, and the Path Reconstructor follows parent links back to the start.

5. Code Level Overview (Level 4)

Class / Function	One-line responsibility
<code>class Maze</code>	Holds the 15x15 grid, start (0,0), goal (14,14); <code>neighbors(cell)</code> returns open N/E/S/W cells.
<code>generate_maze(n)</code>	Builds a perfect maze using the randomized recursive-backtracker.
<code>def bfs(maze)</code>	Queue-based search; returns path and nodes-expanded count.
<code>def dfs(maze)</code>	Stack-based search; same interface as <code>bfs()</code> so the two can be swapped.
<code>reconstruct_path()</code>	Walks the parent map from goal back to start and reverses it.
<code>profile(fn, runs=50)</code>	Times <code>fn</code> with <code>time.perf_counter()</code> and <code>cProfile</code> ; returns best / avg / worst ms.
<code>show_results()</code>	Prints the comparison table and draws the maze with the solved path.

6. Design Decisions

Search logic, maze data and profiling are kept in separate containers so that BFS and DFS can be compared fairly: both get the identical Maze Store and the same Profiling Harness, so any difference in time comes only from the algorithm. Only the Frontier differs between BFS and DFS, which makes the rest of the Search Engine reusable and the comparison clean. No Heuristic Module is included because neither BFS nor DFS uses one; it can be added later as a seventh container if A* is introduced. The maze is generated once and fixed to keep results repeatable.

7. AI Contribution Note

- **AI tools used:** Claude (Anthropic).
- **What AI helped with:** drafting the C4 diagrams and the first version of the text of this report, based on my SLE-2 work and the SLE-3 guideline.
- **What I did myself:** I ran my SLE-2 code on my own machine and verified the timing numbers (BFS and DFS, 50 runs each). I reviewed the code and checked that the container, component and function names in the C4 diagrams match how my program is actually structured.

8. Conclusion

This SLE showed that architecture explains a program at four zoom levels: Context shows who and what the system talks to, Container shows its main parts, Component opens up one important part, and Code names the real classes and functions. Splitting the maze solver into generator, store, search engine, visited set, profiler and output made it clear why the BFS/DFS comparison in SLE-2 was fair, and where new features such as A* or a larger maze would plug in. I learned that a simple, readable structure with few boxes communicates better than a crowded one.